

CKA: Cluster Maintenance

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

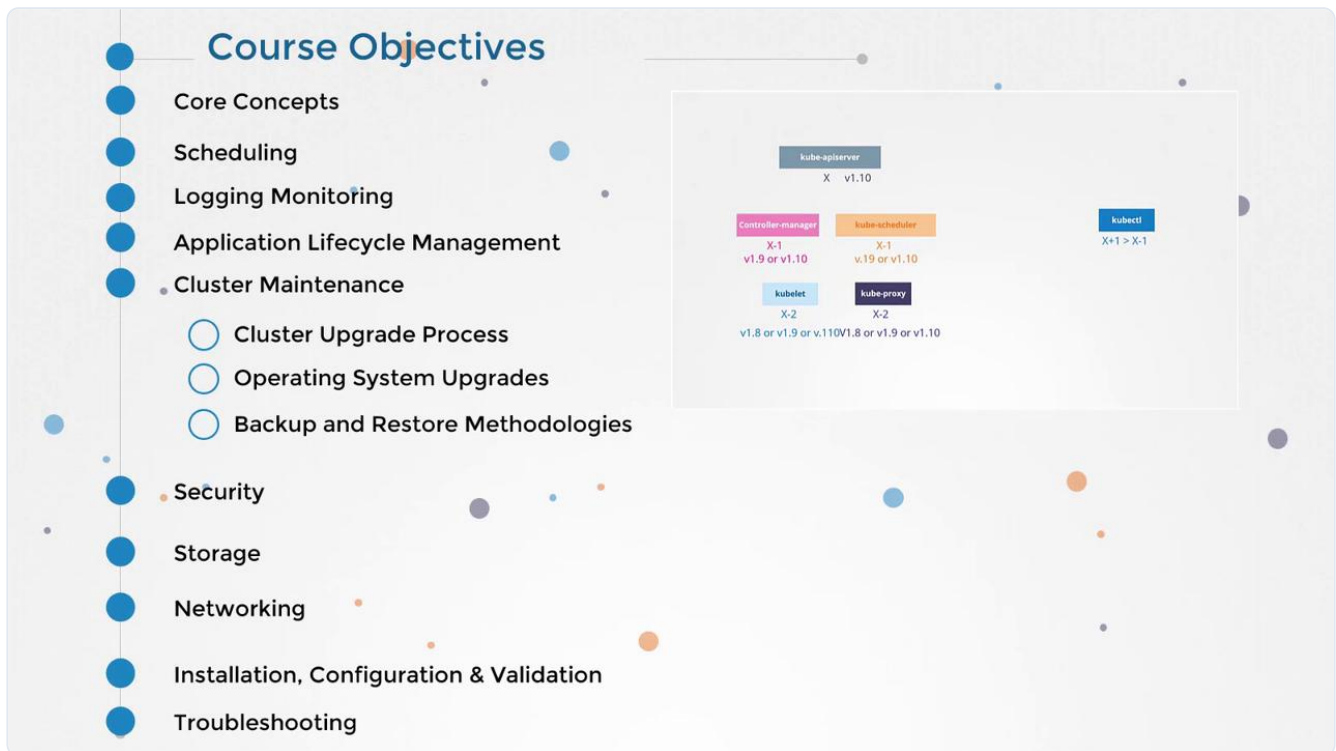
Cluster Maintenance Section Introduction

This article explores essential cluster maintenance topics including operating system upgrades, node removal, cluster upgrade processes, and backup methodologies.

Hello, this is Mumshad Mannambeth. In this article, we will explore essential cluster maintenance topics as part of the [Certified Kubernetes Administrators course](#). We'll cover key areas such as operating system upgrades, node removal, and cluster upgrade processes, as well as backup and disaster recovery methodologies.

Operating System and Node Maintenance

We begin by discussing operating system upgrades and the considerations involved when a node is lost from the cluster, either unintentionally or due to deliberate removal for patching or upgrading purposes. Understanding these procedures is critical to ensuring minimal disruption to your Kubernetes environment.



Cluster Upgrade Process

Before upgrading your cluster, it's important to have a solid grasp of Kubernetes releases, versioning, and best practices for selecting the appropriate upgrade pathway. Once you understand the upgrade procedure, you'll get hands-on experience performing an end-to-end upgrade on a live cluster running applications.

Before proceeding with an upgrade, always run a pre-flight check using the `kubeadm upgrade plan` command. This command helps you verify the cluster's health and fetch available upgrade versions.

To check the upgrade plan, run the following command:

```
``bash theme={null} kubeadm upgrade plan
```

The expected output will look similar to:

```
```bash theme={null}
[upgrade] Running pre-flight checks:
[upgrade] Making sure the cluster is healthy:
[upgrade] Config: Making sure the configuration is correct:
[upgrade] Fetching available versions to upgrade to:
[upgrade/versions] Cluster version: v1.11.1
[upgrade/versions] Kubernetes version: v1.11.8
[upgrade/versions] Latest stable version: v1.11.3
```

Components that must be upgraded manually after you have upgraded the control plane with `kubeadm upgrade apply`:

COMPONENT	CURRENT	AVAILABLE
Kubelet	v1.11.1	v1.11.3

Upgrade to the latest stable version:

COMPONENT	CURRENT	AVAILABLE
API Server	v1.11.8	v1.11.3
Controller Manager	v1.11.8	v1.11.3
Scheduler	v1.11.8	v1.11.3
Kube Proxy	v1.11.8	v1.11.3
Etc	v3.1.8	N/A

You can now apply the upgrade by executing the following command:

After upgrading the control plane with `kubeadm upgrade apply`, make sure to manually upgrade components like kubelet, as highlighted in the output above.

## Backup and Disaster Recovery

In the final part of this module, we focus on backup and restore methodologies. This section will guide you through a disaster recovery simulation where you back up your Kubernetes cluster, simulate a catastrophic event, and then restore the cluster to its original state. This practical exercise is designed to enhance your ability to manage and recover a Kubernetes cluster effectively.

Let's get started on ensuring your clusters are resilient and well-maintained!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/l1ms.txt> Use this file to discover all available pages before exploring further.

# OS Upgrades

This lesson covers handling node upgrades in Kubernetes, focusing on draining, cordoning, and minimizing service interruptions during maintenance or upgrades.

Welcome to this lesson on handling node upgrades in a Kubernetes cluster. In this guide, we will explore various scenarios that require taking a node offline, such as routine maintenance, base software upgrades, or the application of security patches. We also review options to minimize service interruptions during these events.

Imagine a cluster with several nodes where each node runs pods serving your applications. What happens if a node goes down? The pods on that node become inaccessible, and depending on your deployment strategy, this could impact your users. For instance, if you have multiple replicas of a "blue" pod, your users will still be served by the remaining replicas. However, if a "green" pod is the only instance running, its unavailability results in downtime.

Kubernetes offers mechanisms to handle such situations:

- If a node comes back online quickly, the pods are simply restarted.
- If a node remains down for more than five minutes, Kubernetes marks the pods on that node as dead. For pods managed by a ReplicaSet, new pods are automatically created on other nodes. In contrast, pods that are not part of a ReplicaSet will not be restarted, potentially leading to downtime.

When a node is down for a short duration (less than five minutes) and your workload has multiple replicas, a quick upgrade and reboot might be acceptable.

## Draining a Node

For situations where the recovery time is uncertain, the safer method is to drain the node. Draining involves gracefully terminating the pods running on that node so they are recreated on other nodes. At the same time, draining marks the node as unschedulable (cordoned), preventing new pods from being scheduled on it until explicitly allowed.

After the node's workloads have been safely relocated, you can reboot the node. When it comes back online, it remains unschedulable until you uncordon it, allowing new pods to be scheduled. It is important to note that pods moved to other nodes do not automatically revert to the original node after uncordoning. If they were deleted or if new pods have been scheduled across the cluster, Kubernetes maintains the current distribution.

To perform these operations, run the following commands:

```
bash theme={null} kubectl drain node-1 kubectl uncordon node-1
```

## Cordon vs. Drain

---

In addition to draining and uncordoning, Kubernetes provides the cordon command. Unlike drain, cordon only marks a node as unschedulable without terminating or relocating the currently running pods. This ensures that no new pods will be scheduled on the node.

Be cautious when using cordon on a node with critical workloads. Since existing pods remain and new pods cannot be scheduled, your application might become overloaded or experience unexpected behavior.

That concludes this lesson on managing node upgrades in a Kubernetes cluster. You are encouraged to practice these techniques in a hands-on lab or test environment to reinforce your understanding of draining, cordoning, and uncordoning nodes.

Keep building your skills with these essential Kubernetes operations!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Cluster Upgrade Introduction

This article explains the process of upgrading a Kubernetes cluster, focusing on control plane components and strategies for upgrading master and worker nodes.

Welcome to this article on upgrading a Kubernetes cluster. Here, we will explain the upgrade process focusing on the core control plane components, while keeping external dependencies like ETCD and CoreDNS aside. In a previous article, we discussed how Kubernetes manages software releases and how its components can have independent versioning.

It is important to note that not all components are required to run on the same version. Although different components can operate on varying release versions, the Kube API Server remains the primary control plane component that all others communicate with. Consequently, no component should ever run on a version higher than the API Server. For example:

- The controller manager and scheduler may be one version lower than the API Server.
- The Kubelet and Kube Proxy components may be two versions lower than the API Server.

For instance, if the API Server is at version 1.10, then:

- The controller manager and scheduler can run on version 1.10 or 1.9.
- The Kubelet and Kube Proxy can run on version 1.8. Running any component on a version higher than the API Server (e.g., 1.11 when the API Server is 1.10) is not recommended.

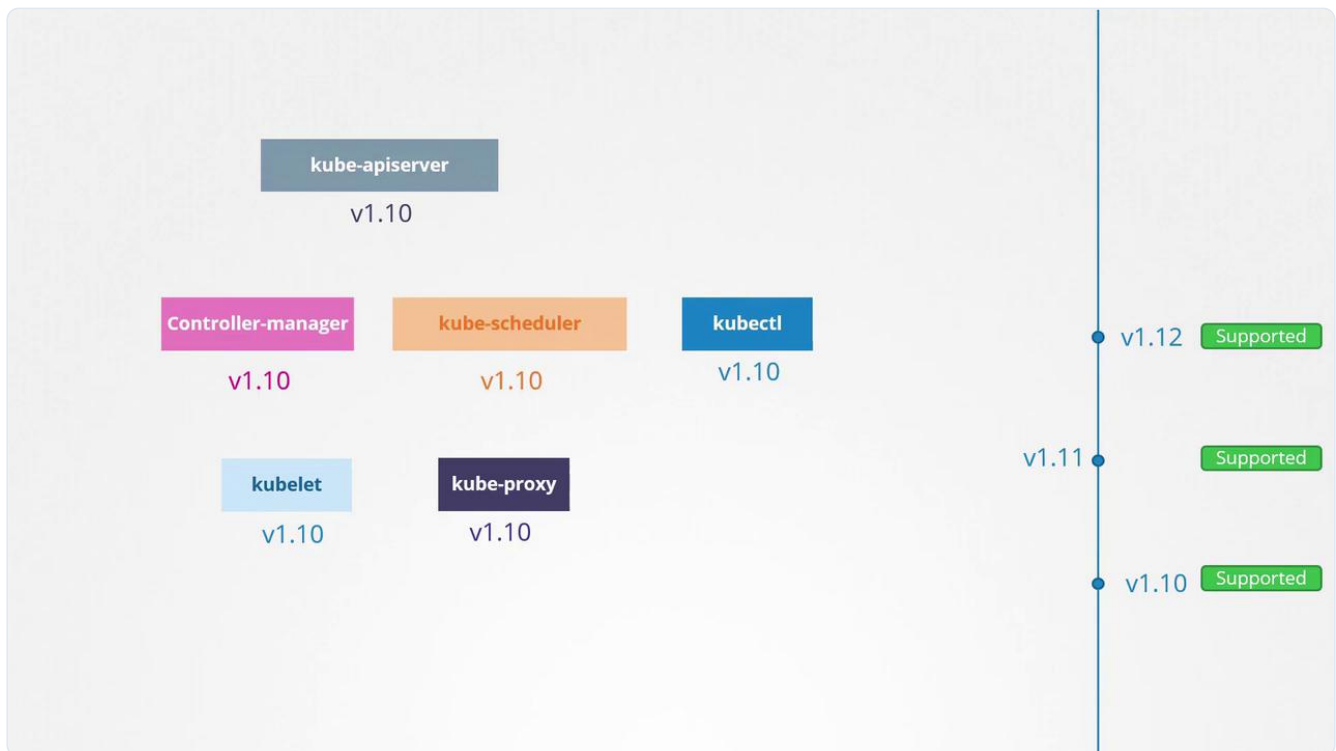
The Kube Control utility is an exception and may run on a version that is higher, lower, or the same as the API Server. This flexibility supports live, rolling upgrades where components can be upgraded individually.

## When to Upgrade

Suppose you're running Kubernetes 1.10, and new releases 1.11 and 1.12 are available. Kubernetes officially supports up to the three most recent minor versions. With 1.12 as the latest, the supported versions are 1.12, 1.11, and 1.10. When version 1.13 is released, only 1.13, 1.12, and 1.11 will be supported. It is advisable to upgrade your cluster to the next release before support for your current version is dropped.

An effective upgrade strategy is to upgrade one minor version at a time (e.g., upgrade from 1.10 to 1.11, then from 1.11 to 1.12, and finally from 1.12 to 1.13) rather than attempting a large jump between versions. Keep in mind that the upgrade process may vary depending on your cluster setup. Managed Kubernetes services (such

as Google Kubernetes Engine) offer a simple upgrade interface, while clusters deployed using tools like kubeadm or manual installation require more hands-on management.



## Upgrade Process Overview

Consider a production cluster with master and worker nodes running version 1.10. The upgrade process generally involves two major steps:

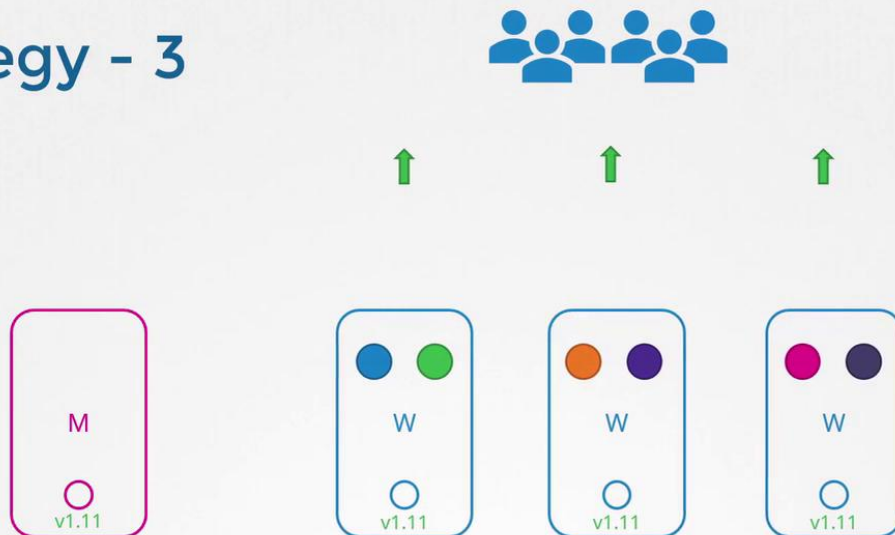
1. Upgrading the master nodes.
2. Upgrading the worker nodes.

During the master node upgrade, control plane components (such as the API server, scheduler, and controller managers) experience a brief interruption. Although management functionality (like kubect! commands or scaling deployments) is paused, the worker nodes continue to run and deliver applications. However, keep in mind that if any pods fail during this period, they might not be restarted automatically. Once the master upgrade is complete, normal control plane operations resume.

After the master nodes are upgraded (for example, moving from version 1.10 to 1.11 while the worker nodes are still at 1.10), the next step is to upgrade the worker nodes. There are several strategies for this:

1. Upgrade all worker nodes simultaneously (which may result in downtime).
2. Upgrade one worker node at a time, allowing workloads to be shifted and ensuring continuous service.
3. Add new nodes with the updated software version, migrate workloads to these new nodes, and then decommission the older nodes.

## Strategy - 3



## Upgrading with kubeadm

Suppose you want to upgrade your cluster from version 1.11 to 1.13. The kubeadm tool simplifies planning and executing cluster upgrades. To start, run:

```
```bash theme={null} kubeadm upgrade plan
```

This command provides useful information such as:

- * The current cluster version.
- * The version of the kubeadm tool.
- * The latest stable version of Kubernetes.
- * A list of control plane components along with their current versions and the target upgrade versions.

Remember that after upgrading the control plane components, you must manually upgrade the kubelet on each node.

Because upgrades are performed one minor version at a time, if you're on version 1.11 and wish to reach 1.13, you must perform two upgrades.

1. Upgrade the kubeadm tool to version 1.12.
2. Upgrade the cluster using the command provided by the upgrade plan (e.g., "kubeadm upgrade apply v1.12.0").

Below is an example of upgrading the control plane components using kubeadm:

```
```bash theme={null}
apt-get upgrade -y kubeadm=1.12.0-00
kubeadm upgrade apply v1.12.0
Output:
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.12.0". Enjoy!
[upgrade/kubelet] Now that your control plane is upgraded, please proceed with upgrading your kubelets
```

After this step, the control plane components are updated to 1.12. Note that if you run:

```
```bash theme={null} kubectl get nodes
```

```
the master node might still appear as 1.11 because this command displays the version of the kubelet registered on the node.  
```bash theme={null}  
kubectl get nodes
```

you should see the master node running 1.12 while the worker nodes continue on 1.11.

## Upgrading Worker Nodes

---

Upgrading worker nodes should be done one at a time. The recommended approach is to drain one worker node to ensure the applications remain available on other nodes. Use the following command to cordon the node and safely evict pods:

```
```bash theme={null} kubectl drain node-1
```

```
After draining the node, upgrade the kubeadm and kubelet packages on the worker node using the same process as the master node.  
```bash theme={null}  
kubectl uncordon node-1
```

Repeat this procedure for each worker node until the entire cluster is updated. When upgrading from 1.11 to 1.12 and eventually to 1.13, follow this process consistently.

## Conclusion

---

In this article, we outlined the process of upgrading a Kubernetes cluster using kubeadm, focusing on two key scenarios: upgrading the control plane (master nodes) and safely upgrading worker nodes without affecting application availability. For best practices, consider testing the upgrade process in a non-production environment to simulate live upgrades and ensure zero downtime for your users.

Good luck with your Kubernetes cluster upgrade!

For more information on Kubernetes best practices and upgrade strategies, please refer to the [Kubernetes Documentation](#).

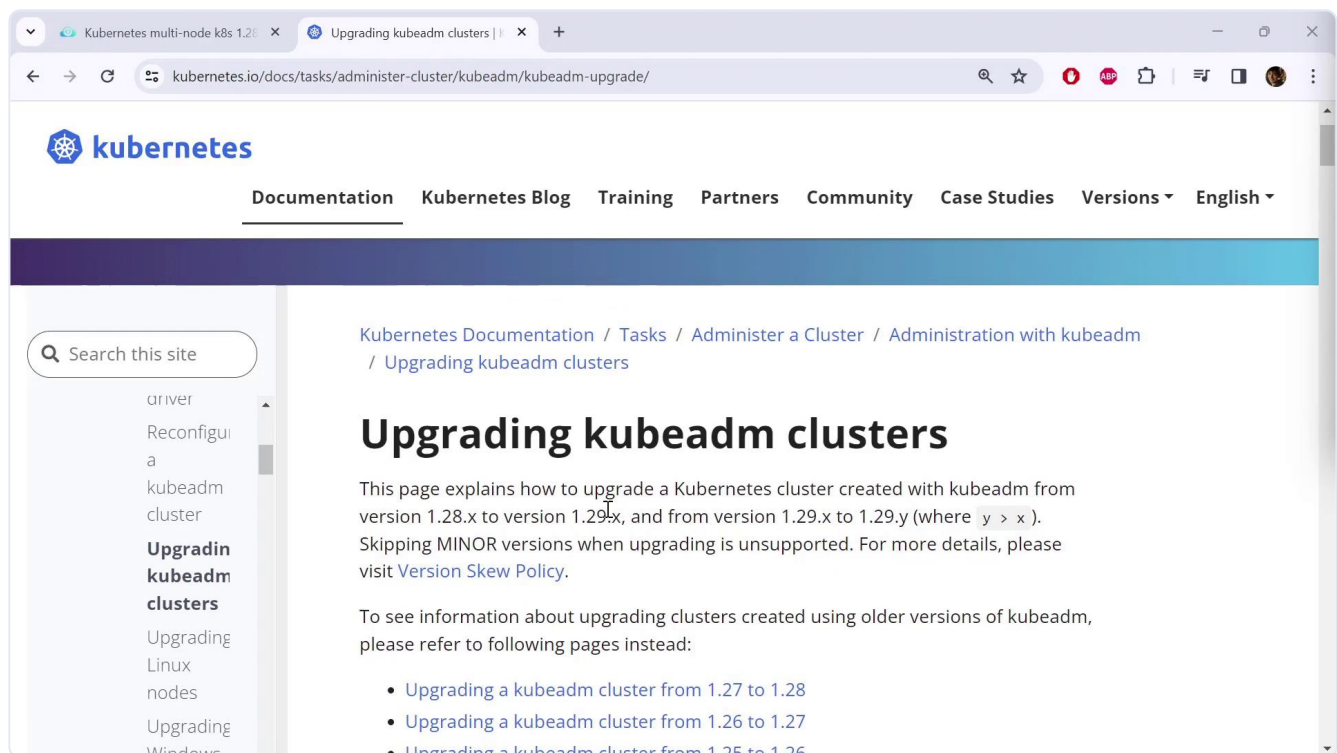
## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Demo Cluster upgrade

This guide demonstrates upgrading a Kubernetes cluster from version 1.28 to 1.29 using kubeadm following official instructions.

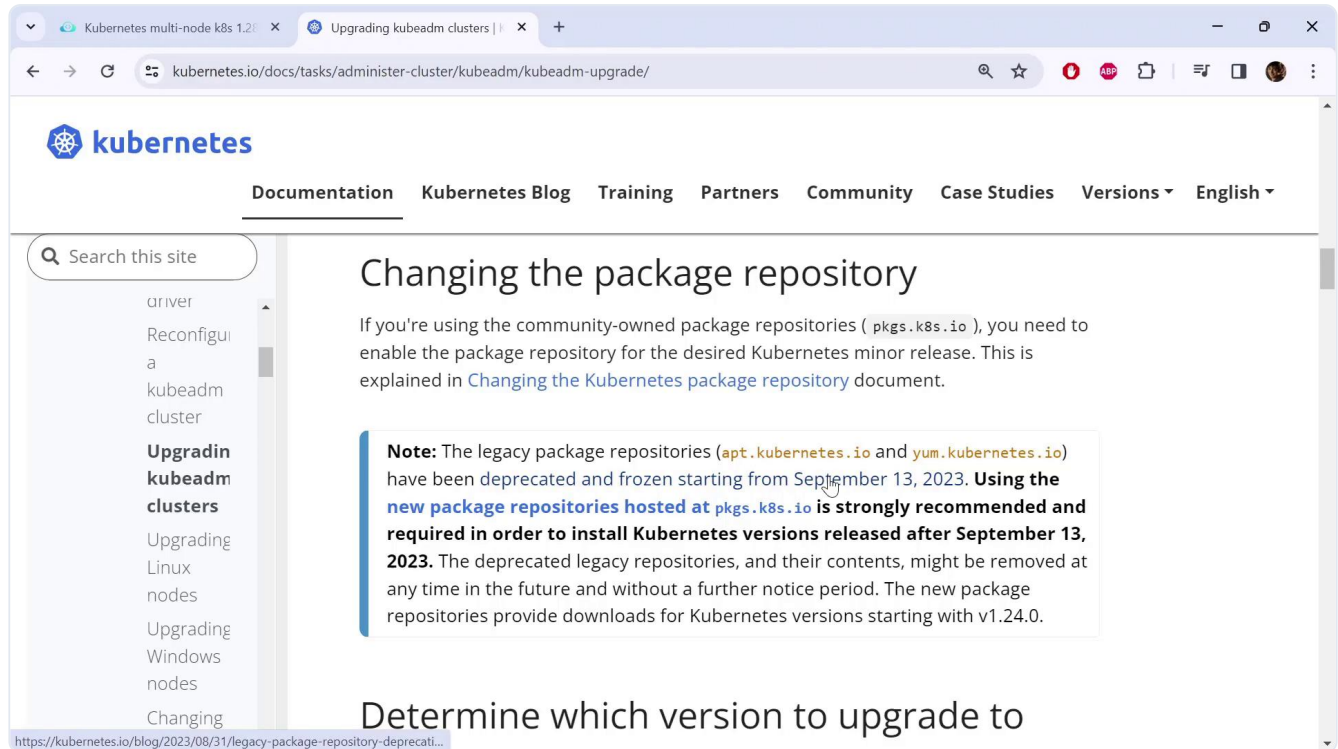
In this guide, we demonstrate how to upgrade a Kubernetes cluster from version 1.28 to 1.29 using kubeadm. We follow the official "Upgrading a Kubeadm Cluster" instructions from the Kubernetes documentation. For further details, refer to the "Upgrading a Kubeadm Cluster" section in the [Kubernetes Documentation](#).



The documentation provides dedicated instructions for each upgrade path. In this case, we are upgrading from 1.28 to 1.29 (the latest release). Similar procedures exist for other upgrade paths (e.g., 1.27 to 1.28, 1.26 to 1.27, etc.). Simply select the correct upgrade path and follow the detailed steps accordingly.

# Updating Package Repositories

Before beginning the upgrade, review the updated package repository information. The legacy repositories (app.kubernetes.io and yum.kubernetes.io) have been deprecated. Moving forward, packages are available at [packages.k8s.io](https://packages.k8s.io). This repository now hosts the latest versions of essential tools such as kubectl and kubeadm.



## Determine which version to upgrade to

Before proceeding, confirm that you are using the new package repositories. Check the documentation and click the provided link for the latest instructions.

## Verify Your Node OS and Repository Setup

First, determine your node distribution because the commands differ slightly by operating system. For example, on a two-node cluster, run:

```
```bash theme={null} kubectl get nodes
```

An example output may be:

```
```plaintext theme={null}
NAME STATUS ROLES AGE VERSION
controlplane Ready control-plane 98m v1.28.0
node01 Ready <none> 98m v1.28.0
```

To check your OS distribution, use:

```
```bash theme={null} cat /etc/release
```

Example output for Ubuntu 20.04.6 LTS (Debian-based):

```
``plaintext theme={null}
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=20.04
DISTRIB_CODENAME=focal
DISTRIB_DESCRIPTION="Ubuntu 20.04.6 LTS"
NAME="Ubuntu"
VERSION="20.04.6 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04"
VERSION_ID="20.04"
...
```

For Debian/Ubuntu systems, update the repository to the new endpoint. First, switch to the new URL for your version upgrade. For version 1.29, execute:

```
``bash theme={null} echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core/stable/v1.29/deb/ " | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

Next, download the public signing key:

```
``bash theme={null}
curl -fsSL https://pkgs.k8s.io/core/stable/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/
```

Remember to adjust the version numbers (e.g., change to 1.29) in both commands as needed during the upgrade process.

After making these updates on the control plane and worker nodes, run:

```
``bash theme={null} sudo apt-get update
```

Upgrading the Control Plane

Step 1: Upgrade kubeadm

On the control plane node, list the available kubeadm versions:

```
``bash theme={null}
sudo apt update
sudo apt-cache madison kubeadm
```

A typical output may appear as follows:

```
``plaintext theme={null} kubeadm | 1.29.3-1.1 | https://pkgs.k8s.io/core/stable/v1.29/deb Packages kubeadm |
1.29.2-1.1 | https://pkgs.k8s.io/core/stable/v1.29/deb Packages kubeadm | 1.29.1-1.1 | https://pkgs.k8s.io/core/
stable/v1.29/deb Packages kubeadm | 1.29.0-1.1 | https://pkgs.k8s.io/core/stable/v1.29/deb Packages
```

Select the latest version (in this example, 1.29.3-1.1) and upgrade kubeadm with:

```
```bash theme={null}
sudo apt-mark unhold kubeadm && \
sudo apt-get update && \
sudo apt-get install -y kubeadm='1.29.3-1.1' && \
sudo apt-mark hold kubeadm
```

Verify that the kubeadm upgrade is successful by checking the version:

```
```bash theme={null} kubeadm version
```

Expected output:

```
```plaintext theme={null}
kubeadm version: &version.Info{Major:"1", Minor:"29", GitVersion:"v1.29.3", ...}
```

## Step 2: Run the Upgrade Plan

Before applying the upgrade, conduct a dry run to ensure compatibility:

```
```bash theme={null} sudo kubeadm upgrade plan
```

This command displays the upgrade details, including the components automatically upgraded and those requiring manual intervention.

```
```plaintext theme={null}
[upgrade/versions] Cluster version: v1.28.0
[upgrade/versions] kubeadm version: v1.29.3
[upgrade/versions] Target version: v1.29.3
...
Components that must be upgraded manually after you upgrade the control plane:
COMPONENT CURRENT TARGET
kubelet 2 x v1.28.0 v1.28.8
...
[upgrade/versions] Please refer to the upgrade process at https://kubernetes.io/docs/tasks/administer-cluster/upgrade-control-planes/ for more information.
```

Kubeadm upgrades most of the control plane components automatically but leaves the kubelet for manual upgrade.

## Step 3: Apply the Control Plane Upgrade

Upgrade your control plane with:

```
```bash theme={null} sudo kubeadm upgrade apply v1.29.3
```

Monitor the process as it renews certificates, updates static Pod manifests, and applies configuration changes.

```
```plaintext theme={null}
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.29.3". Enjoy!
[upgrade/kubelet] Now that your control plane is upgraded, please proceed with upgrading your kubelets.
```

Note: Until kubelet is upgraded, `kubectl get nodes` will still display version v1.28.0 for the kubelet.

## Upgrading kubelet and kubectl on the Control Plane

---

### Step 1: Drain the Control Plane Node

Before updating kubelet and kubectl, drain the control plane node:

```
```bash theme={null} kubectl drain controlplane --ignore-daemonsets
```

```
### Step 2: Upgrade Packages
```

Next, upgrade kubelet and kubectl to version 1.29.3-1.1:

```
```bash theme={null}
sudo apt-mark unhold kubelet kubectl && \
sudo apt-get update && \
sudo apt-get install -y kubelet='1.29.3-1.1' kubectl='1.29.3-1.1' && \
sudo apt-mark hold kubelet kubectl
```

Reload the systemd configuration and restart the kubelet service:

```
```bash theme={null} sudo systemctl daemon-reload sudo systemctl restart kubelet
```

Finally, uncordon the control plane node to allow pods to be scheduled:

```
```bash theme={null}
kubectl uncordon controlplane
```

Verify the upgrade by checking the node versions:

```
```bash theme={null} kubectl get nodes
```

The control plane should now reflect version v1.29.3.

Upgrading Worker Nodes

For each worker node, perform the following steps:

1. **Upgrade kubeadm:**

```
```bash theme={null}
sudo apt-mark unhold kubeadm && \
sudo apt-get update && \
sudo apt-get install -y kubeadm='1.29.3-1.1' && \
sudo apt-mark hold kubeadm
```
```

2. **Update Node Configuration:**

Run the command on the control plane to update the node configuration:

```
```bash theme={null}
sudo kubeadm upgrade node
```
```

This command refreshes the node configuration without upgrading the kubelet package.

3. **Drain the Worker Node:**

Use the node name (e.g., node01) to drain it:

```
```bash theme={null}
kubectl drain node01 --ignore-daemonsets
```
```

4. **Upgrade kubelet and kubectl:**

Execute the following commands on the worker node:

```
```bash theme={null}
sudo apt-mark unhold kubelet kubectl && \
sudo apt-get update && \
sudo apt-get install -y kubelet='1.29.3-1.1' kubectl='1.29.3-1.1' && \
sudo apt-mark hold kubelet kubectl
```
```

Reload and restart kubelet:

```
```bash theme={null}
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```
```

5. **Uncordon the Worker Node:**

```
```bash theme={null}
kubectl uncordon node01
```
```

```
After upgrading all worker nodes, verify that every node in the cluster is running version v1.29.3:

```bash
theme={null}
kubectl get nodes
```

---

## Summary

---

This guide detailed the process of upgrading a Kubernetes cluster using kubeadm by:

1. Updating package repositories to the new [packages.k8s.io](https://packages.k8s.io).
2. Upgrading kubeadm on the control plane and verifying the available versions.
3. Running a dry-run upgrade plan to check compatibility.
4. Applying the control plane upgrade.
5. Upgrading kubelet and kubectl on both the control plane and worker nodes while minimizing downtime by draining and uncordoning nodes.

Following these steps will ensure that your cluster components run on v1.29.3 securely and efficiently.

Happy upgrading!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Backup and Restore Methods

This guide covers backup and restore strategies for Kubernetes environments, focusing on securing deployments and critical components like etcd.

Welcome to this guide on backup and restore strategies for Kubernetes environments. In this lesson, you'll learn how to secure your Kubernetes deployments by backing up declarative configurations, imperative resource changes, and critical cluster components such as etcd.

## What to Back Up

For most Kubernetes deployments, consider backing up:

- **Declarative Configuration Files:** Files defining resources like Deployments, Pods, and Services.
- **Cluster State:** Information stored in the etcd cluster.
- **Imperative Objects:** Resources created on the fly (e.g., namespaces, secrets, configMaps) which might not be documented in files.

Using a declarative approach — creating definition files and applying them with kubectl — not only documents your configuration but also makes it reusable and shareable. For example, here's a simple Pod definition:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod labels: app: myapp type: front-end spec: containers: - name: nginx-container image: nginx
```

Apply the Pod definition with:

```
```bash theme={null}
kubectl apply -f pod-definition.yml
```

Storing your configuration files in a version-controlled repository (such as GitHub) ensures you can quickly restore and redeploy your applications if needed.

## Imperative vs. Declarative Backup Approaches

While the declarative method is preferred, sometimes resources are created using imperative commands. These changes might not be stored in your version control system, which can lead to gaps in your backups. To capture all configurations, you can query the Kubernetes API server directly.

For instance, back up all resources across every namespace by running:

```
```bash theme={null} kubectl get all --all-namespaces -o yaml > all-deploy-services.yaml
```

This command generates a comprehensive YAML snapshot of pods, deployments, services, and other resources.

Backing Up the etcd Cluster

The etcd cluster is the backbone of your Kubernetes system, storing critical state and configuration data.

Below is an example of how etcd might be configured on a master node:

```
```bash theme={null}
ExecStart=/usr/local/bin/etcd \\\
 --name ${ETCD_NAME} \\\
 --cert-file=/etc/etcd/kubernetes.pem \\\
 --key-file=/etc/etcd/kubernetes-key.pem \\\
 --peer-cert-file=/etc/etcd/kubernetes.pem \\\
 --peer-key-file=/etc/etcd/kubernetes-key.pem \\\
 --trusted-ca-file=/etc/etcd/ca.pem \\\
 --peer-trusted-ca-file=/etc/etcd/ca.pem \\\
 --client-cert-auth \\\
 --initial-advertise-peer-urls https://${INTERNAL_IP}:2380 \\\
 --listen-peer-urls https://${INTERNAL_IP}:2380 \\\
 --advertise-client-urls https://${INTERNAL_IP}:2379 \\\
 --initial-cluster etcd-cluster-0 \\\
 --initial-cluster-token etcd-cluster-0 \\\
 --initial-cluster controller-0=https://${CONTROLLER0_IP}:2379 \\\
 --initial-cluster-state new \\\
 --data-dir=/var/lib/etcd
```

etcd offers a built-in snapshot feature via the etcdctl command. To create a snapshot called "snapshot.db", run:

```
```bash theme={null} ETCDCTL_API=3 etcdctl snapshot save snapshot.db
```

After creating the snapshot, you can verify its existence:

```
```bash theme={null}
ls
```

And check the snapshot status:

```
```bash theme={null} ETCDCTL_API=3 etcdctl snapshot status snapshot.db
```

Restoring from an etcd Backup

In the event of a failure, restoring your cluster from an etcd backup involves several steps:

1. ****Stop the Kubernetes API Server:**** The restore process requires stopping the API server.
2. ****Restore the Snapshot:**** Restore the snapshot to a new data directory (e.g., `/var/lib/etcd-from-backup`).

```
```bash theme={null}
ETCDCTL_API=3 etcdctl snapshot restore snapshot.db \
--data-dir /var/lib/etcd-from-backup
```
```

This command initializes a new etcd data directory and reinitializes cluster members.

3. ****Update etcd Configuration:**** Modify your etcd configuration file to point to the new data directory.
4. ****Restart Services:**** Reload the system daemon, restart the etcd service, and finally restart the Kubernetes API server.

<Callout icon="lightbulb" color="#1CB2FE">

Always supply the required certificate files (CA certificate, etcd server certificate, and key) during the restore process.

For an authenticated backup, use:

```
```bash theme={null}
ETCDCTL_API=3 etcdctl snapshot save snapshot.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/etcd/ca.crt \
--cert=/etc/etcd/etcd-server.crt \
--key=/etc/etcd/etcd-server.key
```
```

Choosing the Right Backup Approach

Depending on your environment, the backup strategy might vary:

| Backup Approach | Use Case | Command Example |
|---------------------------------|--|---|
| Declarative File Backup | When configurations are maintained as code | <code>kubectl apply -f pod-definition.yml</code> |
| API Server Configuration Backup | Capturing all cluster resources imperatively | <code>kubectl get all --all-namespaces -o yaml > all-deploy-services.yaml</code> |
| etcd Snapshot | Backing up the critical cluster state | <code>ETCDCTL_API=3 etcdctl snapshot save snapshot.db</code> |

For managed Kubernetes environments where you might not have direct etcd access, relying on API queries is often the more practical solution.

Thank you for following this guide. By mastering these backup and restore strategies, you can enhance the resilience of your Kubernetes clusters and ensure business continuity.